

Regresión Ordinary Least Squares (OLS)

OLS (Ordinary Least Squares) es un método de estimación utilizado en la regresión lineal para encontrar los coeficientes que minimizan la suma de los cuadrados de las diferencias entre los valores observados y los predichos. En otras palabras, OLS busca la "línea de mejor ajuste" a través de los datos, minimizando la suma de los cuadrados de las diferencias verticales entre los valores observados y los predichos.

En el contexto de la regresión lineal, el modelo OLS puede expresarse matemáticamente de la siguiente manera para un conjunto de datos con n observaciones y p variables predictoras:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p + \varepsilon$$

Donde:

- Y es la variable dependiente.
- $X_1, X_2, \dots, X_p$ son las variables predictoras.
- $\beta_0, \beta_1, \dots, \beta_p$ son los coeficientes a estimar.
- ε es el término de error.

El método OLS busca encontrar los valores de $(\beta_0, \beta_1, \dots, \beta_p)$ que minimizan la función de costo, que es la suma de los cuadrados de las diferencias entre los valores observados y los predichos:

$$\text{Minimizar } \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}))^2$$

En este contexto, OLS utiliza técnicas matriciales para encontrar una solución analítica para los coeficientes que minimizan la suma de los cuadrados de las diferencias. Este método es conocido como "mínimos cuadrados ordinarios" porque minimiza la suma de los cuadrados de los residuos.

Diferencia entre OLS y LinearRegression

La diferencia principal entre `statsmodels.OLS` y `sklearn.LinearRegression` radica en su enfoque, funcionalidades y salidas:

1. Enfoque y propósito

- `statsmodels.OLS` :
 - Está diseñado para **análisis estadísticos detallados**.
 - Proporciona **información completa sobre el modelo**, como p-valores, intervalos de confianza, tests de hipótesis (t-test, F-test), R^2 ajustado, y diagnósticos de regresión (heterocedasticidad, multicolinealidad, etc.).
 - Es más utilizado en **econometría, ciencias sociales e investigación** donde la inferencia estadística es importante.
 - No incluye regularización (L1/L2) por defecto.
- `sklearn.LinearRegression` :

- Está optimizado para **predicción y machine learning**.
- Proporciona menos detalles estadísticos (solo coeficientes y R^2 básico).
- Se integra mejor en flujos de trabajo de ML (pipelines, validación cruzada, búsqueda de hiperparámetros).
- No calcula p-valores ni intervalos de confianza por defecto.

2. Formato de entrada

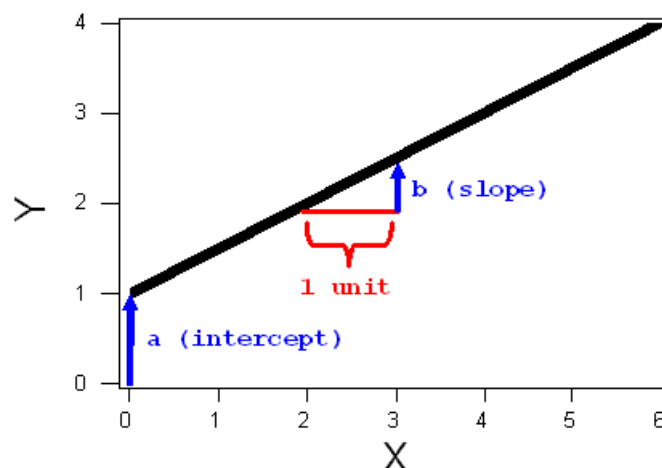
- `statsmodels.OLS` :

- **No añade automáticamente el intercepto (término constante)**. Debes hacerlo manualmente con `sm.add_constant(X)` :

```
import statsmodels.api as sm
X_train_const = sm.add_constant(X_train) # Añade una columna de 1's para el intercepto
model = sm.OLS(y_train, X_train_const).fit()
```

- `sklearn.LinearRegression` :

- **Añade automáticamente el intercepto** (puedes desactivarlo con `fit_intercept=False`).



3. Salida y métodos disponibles

- `statsmodels.OLS` :

- Ofrece un **resumen detallado** con `.summary()` :

```
print(model.summary()) # Muestra p-valores, R², tests, etc.
```

- Métodos útiles: `.predict()`, `.conf_int()`, `.t_test()`, `.f_test()`, etc.

- `sklearn.LinearRegression` :

- Solo proporciona coeficientes (`.coef_`) e intercepto (`.intercept_`).
- No hay p-valores ni tests estadísticos sin librerías adicionales (e.g., `scipy`).

4. Rendimiento y escalabilidad

- `statsmodels` es más lento con datasets grandes (no está optimizado para Big Data).
- `sklearn` es más eficiente y escalable (usa NumPy y optimizaciones para ML).

5. Uso en Machine Learning

- `sklearn` se integra mejor con:
 - Validación cruzada (`cross_val_score`).
 - Pipelines (`Pipeline`).
 - Preprocesamiento (`StandardScaler` , `PolynomialFeatures`).
- `statsmodels` es menos usado en ML, pero mejor para diagnóstico de modelos.